ParkSmart — AI Parking Reservation Platform

Enterprise Technical Documentation

Version: 4.0

Generated: May 21, 2026 at 03:24

Classification: Internal / Portfolio

*Full-stack AI parking reservation system featuring LangGraph orchestration,
Retrieval-Augmented Generation, human-in-the-loop admin approval,
MCP tool protocol, and enterprise CI/CD pipeline.*

Table of Contents

1. Introduction

1.1 Project Overview

ParkSmart is an enterprise-grade AI-powered parking reservation platform that combines conversational AI with structured workflow orchestration. The system allows users to query parking information via natural language and make reservations through a guided chatbot flow, while administrators review and approve bookings through a dedicated dashboard.

Metric	Value
Backend Source Files	28
Backend Lines of Code	5,409
Test Cases	161
Dependencies	25
Frontend Framework	Next.js 16 + React 19
AI Orchestration	LangGraph 0.2+

1.2 Business Problem

Urban parking management faces several challenges:

• **Information fragmentation** — Parking availability, pricing, and hours are scattered across static pages that quickly become outdated.

• **Manual reservation processes** — Phone/email bookings are error-prone and unscalable.

• **Lack of intelligent routing** — Users cannot get real-time answers about availability or pricing without human intervention.

• **No approval workflow** — Reservation requests lack structured review, leading to overbooking and conflicts.

1.3 Solution Approach

ParkSmart solves these problems through a multi-layered architecture:

1. **Conversational AI Interface** — Natural language chatbot powered by GPT-4o with RAG for accurate, context-aware responses about parking facilities.

2. **Structured Reservation Collection** — State machine guides users through step-by-step data collection with validation at each step.

3. **Human-in-the-Loop Approval** — Admin dashboard with LLM-assisted review ensures quality control before confirmation.

4. **Real-Time Notifications** — SMTP email service notifies both admins (new requests) and users (approval/rejection).

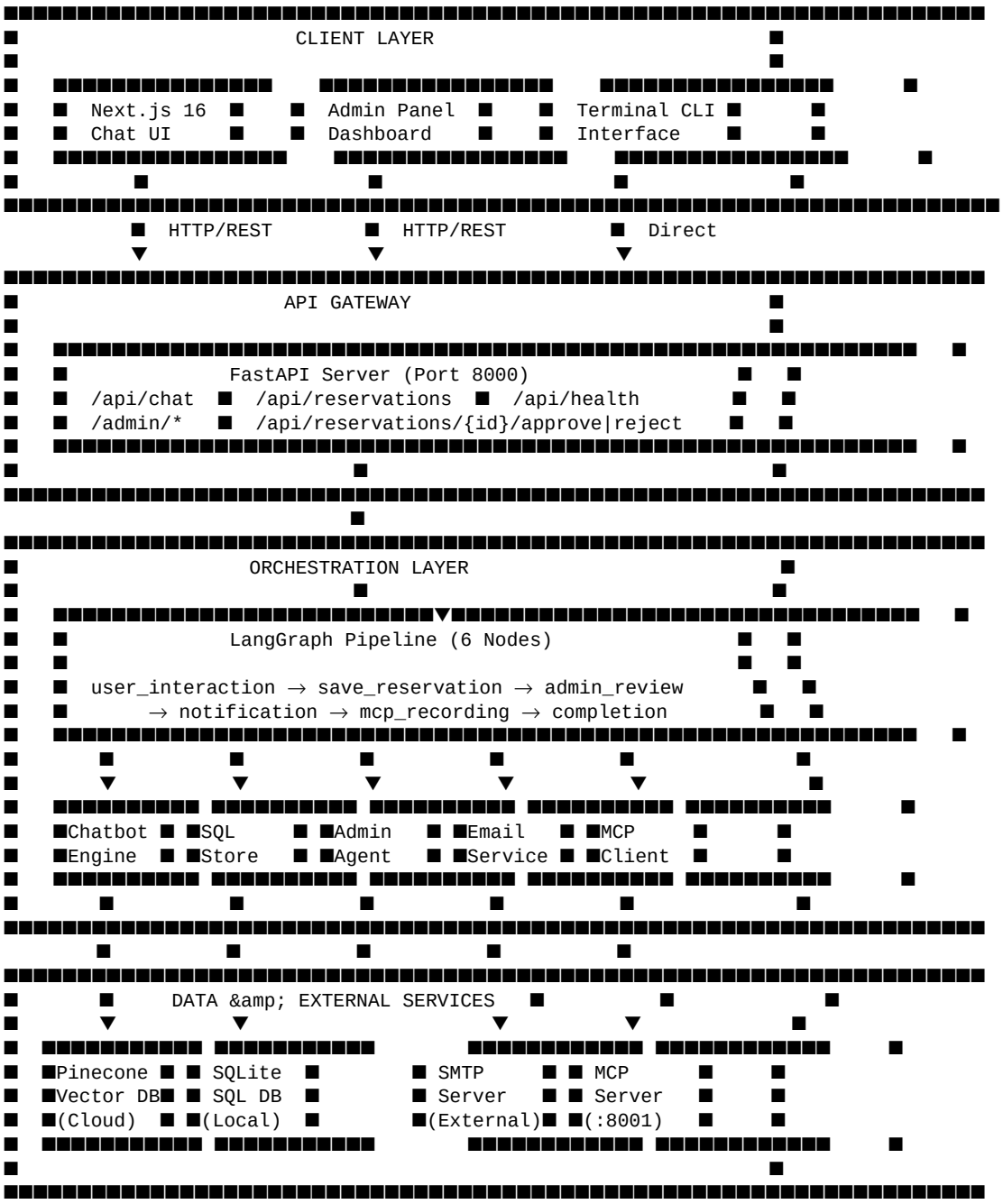
5. **LangGraph Orchestration** — Six-node state graph manages the complete lifecycle from user interaction to completion.

1.4 Key Objectives

- Provide accurate parking information through semantic search (RAG)
- Automate reservation collection with robust input validation
- Enforce human review before confirming any booking
- Maintain audit trail of all reservations and admin actions
- Support both terminal CLI and web-based interfaces
- Implement enterprise patterns: CI/CD, containerization, IaC

2. System Architecture

2.1 High-Level Architecture



2.2 Communication Patterns

Source	Target	Protocol	Purpose
Next.js Frontend	FastAPI Backend	REST/JSON over HTTP	Chat messages, reservation CRUD, admin actions
FastAPI Server	LangGraph Pipeline	In-process function call	Orchestrate reservation lifecycle
Chatbot Engine	Pinecone	HTTPS (gRPC internally)	Vector similarity search for RAG

Chatbot Engine	Azure OpenAI	HTTPS	LLM inference (GPT-4o via EPAM DIAL)
RAG Chain	SQLite	SQLAlchemy ORM	Dynamic context (hours, prices, availability)
Email Service	SMTP Server	SSL/STARTTLS	Notification delivery
MCP Client	MCP Server	HTTP REST	Tool invocation for file recording
Admin Agent	Azure OpenAI	HTTPS	LLM-assisted reservation review

2.3 Data Flow Summary

Dual Database Architecture:

• **Pinecone (Vector DB)** — Stores embeddings of static parking information (facility descriptions, policies, FAQ). Used for semantic retrieval in RAG pipeline.

• **SQLite (Relational DB)** — Stores dynamic transactional data: working hours, pricing tables, space availability, and reservation records with full lifecycle tracking.

This split ensures that semantic search performance is decoupled from transactional write throughput, allowing each datastore to be optimized independently.

3. Technology Choices

3.1 Backend Framework — FastAPI

Criterion	FastAPI	Flask	Django REST
Async Support	Native (ASGI)	Limited (via gevent)	Partial (Django 4.1+)
Auto Docs	Swagger + ReDoc built-in	Manual	DRF Browsable API
Type Safety	Pydantic models enforced	None	Serializers
Performance	High (Starlette/Uvicorn)	Moderate (WSGI)	Moderate
Learning Curve	Low	Very Low	High

Decision: FastAPI provides native async support essential for concurrent LLM calls, automatic OpenAPI documentation for frontend integration, and Pydantic validation that catches malformed requests at the boundary.

3.2 Frontend Framework — Next.js 16

Criterion	Next.js	Create React App	Vite + React
SSR/SSG	Built-in (App Router)	None	Plugin required
Routing	File-based (automatic)	Manual (react-router)	Manual
API Routes	Built-in	None	None
Production Build	Optimized + CDN-ready	Basic	Fast but manual
TypeScript	First-class	Supported	Supported

Decision: Next.js App Router provides file-based routing that maps cleanly to the `/chat` and `/admin` page structure, plus built-in optimization for production deployment.

3.3 Vector Database — Pinecone

Criterion	Pinecone	ChromaDB	Weaviate	FAISS
Managed Service	Yes (serverless)	Self-hosted	Self-hosted	Library only
Scalability	Auto-scaling	Manual	Manual	In-memory
Metadata Filtering	Yes	Yes	Yes	No
Production Ready	Enterprise-grade	Development	Production	Research
Maintenance	Zero-ops	High	High	None (in-process)

Decision: Pinecone's serverless deployment eliminates infrastructure management while providing sub-100ms similarity search at scale. The managed service aligns with the project's cloud-first deployment strategy.

3.4 AI Orchestration — LangGraph

Criterion	LangGraph	LangChain Agents	Custom FSM
State Management	TypedDict (explicit)	AgentState (implicit)	Manual

Graph Visualization	Built-in	None	Manual
Human-in-Loop	Native interrupt	Custom implementation	Custom
Conditional Routing	Declarative edges	Tool-based	if/else chains
Reproducibility	Deterministic graph	Non-deterministic	Deterministic

Decision: LangGraph provides declarative state machine semantics that make the six-node reservation pipeline explicit, testable, and auditable. The conditional edge system cleanly handles branching between booking and Q&A flows.

3.5 Containerization — Docker

Rationale:

- Multi-stage builds reduce image size (builder → runtime separation)
- Non-root user execution for security
- Health checks enable orchestration readiness probes
- `docker-compose.yml` defines the complete local development environment
- Environment variable injection isolates secrets from code

3.6 CI/CD — GitHub Actions

Rationale:

- Native GitHub integration with zero additional infrastructure
- Matrix strategy support for multi-Python-version testing
- Artifact caching (pip, npm) reduces build times by ~60%
- Three parallel workflows: backend tests, frontend validation, Docker build

3.7 Infrastructure as Code — Terraform

Rationale:

- Declarative infrastructure definition prevents configuration drift
- Render provider enables one-command cloud deployment
- Variable files separate sensitive values from infrastructure logic
- State management provides deployment audit trail

4. Folder Structure

4.1 Complete Directory Layout

```
■■■ config/
■   ■■■ __init__.py
■   ■■■ settings.py
■■■ data/
■   ■■■ approved_reservations.txt
■   ■■■ README.md
■■■ frontend/
■   ■■■ public/
■   ■■■ src/
■   ■   ■■■ app/
■   ■   ■   ■■■ admin/
■   ■   ■   ■■■ chat/
■   ■   ■   ■■■ globals.css
■   ■   ■   ■■■ layout.tsx
■   ■   ■   ■■■ page.tsx
■   ■   ■■■ components/
■   ■   ■   ■■■ admin/
■   ■   ■   ■■■ chat/
■   ■   ■   ■■■ providers/
■   ■   ■   ■■■ shared/
■   ■   ■   ■■■ ui/
■   ■   ■■■ hooks/
■   ■   ■   ■■■ useAutoScroll.ts
■   ■   ■   ■■■ useMediaQuery.ts
■   ■   ■■■ lib/
■   ■   ■   ■■■ api.ts
■   ■   ■   ■■■ helpers.ts
■   ■   ■   ■■■ utils.ts
■   ■   ■   ■■■ validations.ts
■   ■   ■■■ services/
■   ■   ■   ■■■ adminService.ts
■   ■   ■   ■■■ chatService.ts
■   ■   ■   ■■■ reservationService.ts
■   ■   ■■■ store/
■   ■   ■   ■■■ adminStore.ts
■   ■   ■   ■■■ authStore.ts
■   ■   ■   ■■■ chatStore.ts
■   ■   ■   ■■■ uiStore.ts
■   ■   ■■■ types/
■   ■       ■■■ index.ts
■   ■■■ .env.example
■   ■■■ .env.local
■   ■■■ AGENTS.md
■   ■■■ CLAUDE.md
■   ■■■ components.json
■   ■■■ Dockerfile
■   ■■■ eslint.config.mjs
■   ■■■ next-env.d.ts
■   ■■■ next.config.ts
■   ■■■ package-lock.json
■   ■■■ package.json
■   ■■■ postcss.config.mjs
■   ■■■ README.md
■   ■■■ tsconfig.json
■   ■■■ tsconfig.tsbuildinfo
■■■ logs/
■   ■■■ access.log
■   ■■■ app.log
■   ■■■ error.log
■■■ src/
■   ■■■ agents/
■   ■   ■■■ __init__.py
■   ■   ■■■ admin_agent.py
■   ■■■ api/
■   ■   ■■■ __init__.py
■   ■   ■■■ server.py
```

```
■   ■■■ chatbot/
■   ■   ■■■ __init__.py
■   ■   ■■■ chatbot.py
■   ■   ■■■ guardrails.py
■   ■   ■■■ rag_chain.py
■   ■■■ data/
■   ■   ■■■ __init__.py
■   ■   ■■■ load_data.py
■   ■   ■■■ parking_info.txt
■   ■■■ database/
■   ■   ■■■ __init__.py
■   ■   ■■■ sql_store.py
■   ■   ■■■ vector_store.py
■   ■■■ evaluation/
■   ■   ■■■ __init__.py
■   ■   ■■■ evaluator.py
■   ■■■ graph/
■   ■   ■■■ __init__.py
■   ■   ■■■ nodes.py
■   ■   ■■■ pipeline.py
■   ■   ■■■ state.py
■   ■■■ mcp/
■   ■   ■■■ __init__.py
■   ■   ■■■ mcp_client.py
■   ■   ■■■ mcp_server.py
■   ■■■ notifications/
■   ■   ■■■ __init__.py
■   ■   ■■■ email_service.py
■   ■■■ utils/
■   ■   ■■■ __init__.py
■   ■   ■■■ logging_config.py
■   ■   ■■■ masking.py
■   ■■■ __init__.py
■■■ terraform/
■   ■■■ main.tf
■   ■■■ outputs.tf
■   ■■■ provider.tf
■   ■■■ variables.tf
■■■ tests/
■   ■■■ __init__.py
■   ■■■ test_admin_agent.py
■   ■■■ test_api_server.py
■   ■■■ test_chatbot.py
■   ■■■ test_email_service.py
■   ■■■ test_evaluator.py
■   ■■■ test_graph.py
■   ■■■ test_guardrails.py
■   ■■■ test_masking.py
■   ■■■ test_mcp.py
■   ■■■ test_sql_store.py
■   ■■■ test_vector_store.py
■■■ .coverage
■■■ .dockerignore
■■■ .env
■■■ .env.example
■■■ .flake8
■■■ AGENTS.md
■■■ coverage.xml
■■■ create_stage4_implementation.py
■■■ create_stage4_presentation.py
■■■ DEPLOYMENT.md
■■■ docker-compose.yml
■■■ Dockerfile
■■■ generate_project_documentation.py
■■■ IMPLEMENTATION_REPORT.md
■■■ main.py
■■■ MIGRATION_CHROMADB_TO_PINECONE.md
■■■ ParkSmart_Stage4_Presentation.pptx
■■■ PROJECT_DOCUMENTATION.md
■■■ pyproject.toml
■■■ pytest.ini
```

```
■■■ README.md
■■■ requirements.txt
■■■ show_tables.py
■■■ test_graph_live.py
■■■ view_db.py
```

4.2 Directory Explanations

Directory	Purpose	Key Contents
`config/`	Application configuration	`settings.py` — Pydantic BaseSettings reading `.env`
`src/`	All backend source code	7 sub-packages covering every system layer
`src/chatbot/`	Conversational AI engine	Chatbot state machine, RAG chain, guardrails
`src/database/`	Data persistence	SQLAlchemy ORM (`sql_store.py`), Pinecone wrapper (`vector_store.py`)
`src/graph/`	LangGraph orchestration	Pipeline builder, node implementations, state schema
`src/agents/`	Admin AI agent	LangChain agent with tools for reservation review
`src/api/`	REST API layer	FastAPI server with chat, reservation, and admin endpoints
`src/notifications/`	Email service	SMTP with retry, HTML/plain templates, async support
`src/mcp/`	Model Context Protocol	Tool server (FastAPI) + client with local fallback
`src/data/`	Static knowledge base	`parking_info.txt` + document loader for RAG
`src/evaluation/`	RAG quality metrics	Precision@K, Recall@K, answer relevance, latency
`src/utils/`	Shared utilities	Email masking, structured logging
`tests/`	Test suite (161 tests)	One test file per module, mocked external services
`frontend/`	Next.js 16 web application	Chat UI, admin dashboard, Zustand stores
`frontend/src/components/`	React components	Chat (3), Admin (5), UI primitives (shadcn)
`frontend/src/store/`	State management	Zustand stores: chat, auth, admin, UI
`frontend/src/services/`	API integration	Axios-based service layer for backend calls
`.github/workflows/`	CI/CD pipelines	Backend CI, Frontend CI, Docker Build validation
`terraform/`	Infrastructure as Code	Render deployment with variables and outputs
`data/`	Runtime data files	`approved_reservations.txt` (MCP output)

5. File-by-File Explanation

This section provides a detailed analysis of every major file in the project, explaining its purpose, internal structure, dependencies, and how it integrates with the rest of the system.

5.1 Entry Points & Configuration

`main.py`

Lines: 443 | **Purpose:** Application entry point supporting multiple execution modes

Serves as the unified launcher for all system components. Supports CLI flags:

- `--setup` — Initialize vector DB and SQL DB with parking data
- `--evaluate` — Run RAG accuracy and latency benchmarks
- `--api` — Start FastAPI REST server on port 8000
- `--mcp` — Start MCP tool server on port 8001
- `--admin` — Launch admin review CLI
- Default — Interactive terminal chatbot

Integration: Instantiates all service objects (chatbot, SQL store, email, MCP client, admin agent) and passes them to the LangGraph pipeline builder.

Functions:

- `setup_databases()` — Initialize and populate the databases.

Run this once when se

- `run_evaluation()` — Run the RAG evaluation suite and print the report.

This mea

- `run_chatbot()` — Start the interactive chatbot in the terminal.

The chatbot

- `run_admin_panel()` — Start the admin panel for reviewing reservations.

This laun

- `run_api_server()` — Start the FastAPI REST API server.

This launches the REST A

- `run_mcp_server()` — Start the MCP (Model Context Protocol) server.

This is the

- `run_graph_pipeline()` — Run the LangGraph-orchestrated pipeline (Stage 4).

This rep

- `main()` — Parse arguments and run the appropriate command.

Key Imports: argparse, config, dotenv, src

`config/settings.py`

Lines: 94 | **Purpose:** Centralized configuration via Pydantic BaseSettings

Reads all configuration from environment variables (`.env` file). Key settings:

- **LLM:** Azure endpoint, API key, model name, temperature
- **Embeddings:** Model name (`all-MiniLM-L6-v2`), dimension (384)
- **Pinecone:** API key, index name, cloud/region

- **SQL:** Database URL (SQLite path)
- **SMTP:** Host, port, credentials, admin email
- **Guardrails:** Enable/disable toggle, confidence threshold

Convention: All modules import `from config.settings import settings`. Never use `os.getenv()` directly — Pydantic handles `.env` loading automatically.

Classes:

- `Settings` — Application settings loaded from environment variables....

Key Imports: pydantic, pydantic_settings

5.2 Chatbot Module (`src/chatbot/`)

`src/chatbot/chatbot.py`

Lines: 443 | **Purpose:** Main chatbot orchestration with finite state machine

Implements a conversation state machine with states:

`IDLE → COLLECTING_NAME → COLLECTING_EMAIL → COLLECTING_CAR → COLLECTING_SPACE_TYPE → COLLECTING_START → COLLECTING_END → CONFIRMING`

Workflow:

1. User message enters `chat()` → guardrails check input
2. If in IDLE state → route to RAG for general queries
3. If RAG response contains `INTENT:BOOKING` → start reservation flow
4. State machine collects fields one-by-one with validation
5. On confirmation → save to SQL, notify admin via email

Integration: Uses VectorStore (RAG), SQLStore (save reservation), EmailService (admin notification), Guardrails (safety filter).

Classes:

- `ConversationState` — Possible states in the conversation flow....
- `ReservationData` — Holds the data collected during the reservation process.

Each field is filled st...

- `is_complete()`
- `to_dict()`
- `summary()`
- `ParkingChatbot` — Main chatbot class that manages the full conversation.

This is the primary inte...

- `__init__()`
- `chat(user_message)`
- `_handle_general_query(message)`
- `_start_reservation()`
- `_handle_reservation_flow(message)`
- `_collect_name(message)`
- `_collect_email(message)`
- `_collect_car(message)`
- `_collect_space_type(message)`
- `_collect_start_time(message)`

- `_collect_end_time(message)`
- `_handle_confirmation(message)`

Key Imports: src

`src/chatbot/rag_chain.py`

Lines: 226 | **Purpose:** Retrieval-Augmented Generation pipeline (LCEL)

Builds a LangChain Expression Language (LCEL) chain:

`retriever → dynamic_context → prompt_template → LLM → output_parser`

Features:

- Retrieves top-K documents from Pinecone via cosine similarity
- Injects real-time SQL data (hours, prices, availability) as dynamic context
- Maintains chat history for multi-turn conversations
- System prompt includes `INTENT:BOOKING` detection instruction
- Uses AzureChatOpenAI via EPAM DIAL proxy

Integration: Called by `ParkingChatbot._handle_general_query()` for all non-reservation messages.

Classes:

- `RAGChain` — The main RAG chain that processes user queries.

Flow:

User Question → Vector Se...

- `__init__(vector_store, sql_store)`
- `_build_chain()`
- `ask(question)`
- `get_relevant_documents(query)`
- `clear_history()`
- `get_retrieval_context(question)`

Key Imports: config, langchain_core, langchain_openai, src

`src/chatbot/guardrails.py`

Lines: 293 | **Purpose:** Security layer for PII detection and prompt injection prevention

Input Protection:

- Regex-based prompt injection detection (ignore instructions, system prompt, etc.)
- Data privacy request detection (show all users, dump database)
- Returns blocked response with explanation

Output Protection:

- Microsoft Presidio NLP engine for PII entity detection
- Regex fallback when Presidio/spaCy unavailable
- Redacts: email addresses, phone numbers, SSNs, credit cards
- Safe patterns list excludes our own contact info from redaction
- Configurable confidence threshold (default 0.7)

Integration: Wraps every `chatbot.chat()` call — input checked before processing, output filtered before returning to user.

Classes:

- `Guardrails` — Guardrails for protecting sensitive data and preventing misuse.

Uses a dual app...

- `__init__()`
- `check_input(user_input)`
- `filter_output(response)`
- `_filter_with_presidio(text, safe_ranges)`
- `_filter_with_regex(text, safe_ranges)`
- `detect_pii_in_text(text)`

Key Imports: config

5.3 Database Module (`src/database/`)

`src/database/sql_store.py`

Lines: 540 | **Purpose:** SQLAlchemy ORM for transactional parking data

ORM Models (4 tables):

- `WorkingHours` — Day-specific operating hours
- `ParkingPrice` — Space type + duration → price mapping
- `ParkingAvailability` — Floor × space type → capacity/available
- `Reservation` — Full lifecycle (pending → approved/rejected)

Key Capabilities:

- Schema migration on startup (`ALTER TABLE` for new columns)
- Default data population (hours, prices, 4-floor availability)
- Dynamic context generation for RAG injection
- StaticPool for in-memory SQLite thread safety

Integration: Used by chatbot (reservation save), admin agent (review/approve), API server (CRUD endpoints), RAG chain (dynamic context).

Classes:

- `WorkingHours` — Stores the opening and closing hours for each day of the week.

Example: Monday -...

- `ParkingPrice` — Stores pricing for different parking space types and durations.

Example: Standar...

- `ParkingAvailability` — Stores current availability of parking spaces by type and floor.

Example: Floor ...

- `Reservation` — Stores parking reservation requests submitted by users.

Lifecycle: pending → ap...

- `SQLStore` — Manages the SQL database for dynamic parking data.

Provides methods to:

- Initi...
- `__init__(database_url)`
- `_migrate_schema()`
- `initialize_default_data()`

- `get_working_hours()`
- `get_prices(space_type)`
- `get_availability(space_type, floor)`
- `get_total_availability()`
- `get_dynamic_context()`
- `save_reservation(reservation_data)`
- `get_reservations(status)`
- `get_reservation_by_id(reservation_id)`
- `update_reservation_status(reservation_id, status, admin_notes)`

Key Imports: config, sqlalchemy

`src/database/vector_store.py`

Lines: 298 | **Purpose:** Pinecone vector database wrapper for semantic search

Capabilities:

- Initializes Pinecone serverless index (384-dim, cosine metric, AWS us-east-1)
- Automatic index creation with readiness polling
- Batch document upsert with configurable batch size
- Similarity search with score (returns documents + cosine distances)
- Async variants for all search methods
- LangChain retriever interface for LCEL integration
- Connection validation and health checks
- Retry with exponential backoff via tenacity

Integration: Provides retriever to RAG chain, loaded by `load_data.py` during setup.

Classes:

- `VectorStoreError` — Base exception for vector store operations....
- `VectorStoreConnectionError` — Raised when connection to Pinecone fails....
- `VectorStore` — Wrapper around Pinecone for storing and retrieving parking information.

This cl...

- `__init__()`
- `_ensure_index()`
- `_wait_for_index_ready(timeout)`
- `validate_connection()`
- `add_documents(documents, batch_size)`
- `similarity_search(query, k)`
- `similarity_search_with_score(query, k)`
- `async asimilarity_search(query, k)`
- `async asimilarity_search_with_score(query, k)`
- `get_retriever(search_kwargs)`
- `get_collection_count()`
- `clear()`

Key Imports: config, langchain_core, langchain_huggingface, langchain_pinecone, pinecone, tenacity, time, uuid

5.4 Graph Orchestration Module (`src/graph/`)

`src/graph/state.py`

Lines: 94 | **Purpose:** TypedDict schema defining the LangGraph state contract

GraphState Fields:

- `user_message`, `bot_response` — Current turn I/O
- `conversation_phase` — Pipeline phase enum
- `reservation_data` — Collected booking information
- `reservation_id` — SQL primary key after save
- `admin_decision`, `admin_notes` — Review outcome
- `notification_sent`, `mcp_recorded` — Completion flags
- `history` — Conversation message list
- `is_booking_flow`, `needs_admin_input`, `admin_input` — Routing signals

PipelinePhase Enum:

`USER_INTERACTION → BOOKING_COMPLETE → AWAITING_ADMIN → ADMIN_REVIEWING → APPROVED/REJECTED → NOTIFYING → RECORDING → COMPLETED | ERROR`

Classes:

- `PipelinePhase` — Tracks where the reservation is in the overall pipeline.

This replaces the old ...

- `GraphState` — The shared state object that flows through every node in the LangGraph.

Every n...

`src/graph/pipeline.py`

Lines: 353 | **Purpose:** LangGraph state machine builder and execution engine

Graph Construction:

1. Creates `StateGraph(GraphState)` with 6 nodes
2. Defines conditional edges for routing:

• After user_interaction → save_reservation (if booking) or END (if Q&A)
• After admin_review → notification (if decided) or END (await input)
• After notification → mcp_recording (if approved) or completion (if rejected)

3. Compiles to executable graph

Execution Functions:

- `run_user_message()` — Process user input through the pipeline
- `run_admin_decision()` — Route admin approval through notification→MCP→completion

Integration: Built in `main.py` and `server.py`, receives all service instances.

Functions:

- `after_user_interaction(state)` — Decide what happens after the user interaction node.
- If a
- `after_admin_review(state)` — Decide what happens after the admin reviews.
- If admin app
- `after_notification(state)` — Decide what happens after notification is sent.
- If approv

• `create_pipeline(chatbot, sql_store, email_service, mcp_client)` — Build and compile the LangGraph state graph.

This function:

• `create_initial_state()` — Create a fresh initial state for a new conversation.

Return

• `run_user_message(pipeline, user_message, current_state)` — Run a user message through the pipeline.

Takes the user's m

• `run_admin_decision(pipeline, current_state, admin_command)` — Run an admin decision through the pipeline.

After a reserva

Key Imports: langgraph, src

`src/graph/nodes.py`

Lines: 457 | **Purpose:** Individual node implementations for the LangGraph pipeline

6 Node Functions:

1. `user_interaction_node()` — Calls chatbot, detects booking completion
2. `save_reservation_node()` — Transitions to AWAITING_ADMIN phase
3. `admin_review_node()` — Parses admin commands (approve/reject/review)
4. `notification_node()` — Updates DB status, sends email notification
5. `mcp_recording_node()` — Writes approved reservation to file via MCP
6. `completion_node()` — Finalizes the pipeline cycle

Singleton Pattern: Module-level variables hold shared service instances, initialized once by `initialize_components()` from pipeline builder.

Functions:

• `initialize_components(chatbot, sql_store, email_service, mcp_client)` — Initialize shared components used by all graph nodes.

This

• `user_interaction_node(state)` — Handle user messages through the chatbot.

This node:

1. Tak

• `_extract_reservation_id(response)` — Extract reservation ID from chatbot response.

The chatbot r

• `save_reservation_node(state)` — After booking is complete, notify admin and transition to aw

• `admin_review_node(state)` — Admin reviews the reservation and makes a decision.

This is

• `notification_node(state)` — Send email notifications based on admin decision.

For APPRO

• `mcp_recording_node(state)` — Write approved reservation to file via MCP server.

This nod

• `completion_node(state)` — Final node — marks the pipeline as completed.

Generates a f

Key Imports: src

5.5 Admin Agent Module (`src/agents/`)

`src/agents/admin_agent.py`

Lines: 467 | **Purpose:** LangChain agent with tools for intelligent reservation review

LangChain Tools (3):

- `check_availability` — Query real-time parking availability by space type
- `list_pending` — Retrieve all pending reservations
- `get_reservation` — Fetch single reservation by ID

Agent Capabilities:

- `review_reservation()` — LLM generates recommendation with availability context
- `approve_reservation()` — Updates status, emails user, writes MCP file
- `reject_reservation()` — Updates status, emails rejection with notes
- `get_admin_summary()` — Dashboard metrics (pending/approved/rejected counts)

CLI Interface: `run_admin_cli()` provides terminal commands: list, all, review, approve, reject, dash, quit.

Classes:

- `AdminAgent` — LangChain-powered admin agent for reviewing parking reservations.

This is the s...

- `__init__(sql_store)`
- `get_pending_reservations()`
- `review_reservation(reservation_id)`
- `approve_reservation(reservation_id, admin_notes)`
- `reject_reservation(reservation_id, admin_notes)`
- `get_admin_summary()`

Functions:

- `create_check_availability_tool(sql_store)` — Create a tool that checks parking space availability.
- `create_get_reservation_tool(sql_store)` — Create a tool that fetches reservation details by ID.
- `create_list_pending_tool(sql_store)` — Create a tool that lists all pending reservations.
- `run_admin_cli()` — Run the admin agent in CLI (terminal) mode.

This provides a

Key Imports: config, langchain_core, langchain_openai, requests, src

5.6 API Server Module (`src/api/`)

`src/api/server.py`

Lines: 486 | **Purpose:** FastAPI REST API for frontend integration and external access

Endpoint Groups:

Method	Path	Purpose
GET	`/api/health`	Basic health check

GET	`/api/health/detailed`	DB + Vector DB + Email status
POST	`/api/chat`	Send message to chatbot pipeline
POST	`/api/reservations`	Create new reservation
GET	`/api/reservations`	List reservations (with status filter)
GET	`/api/reservations/{id}`	Get reservation by ID
PUT	`/api/reservations/{id}/approve`	Admin approve
PUT	`/api/reservations/{id}/reject`	Admin reject

Middleware: CORS configured for `localhost:3000` (frontend dev server)

Note: `_pipeline_state` is global mutable state — single-session demo only, not thread-safe for concurrent requests.

Classes:

- `ReservationRequest` — JSON body for creating a new reservation (sent by chatbot)....
- `ReservationResponse` — JSON response when returning reservation data....
- `AdminActionRequest` — JSON body for admin approve/reject action....
- `StatusResponse` — Generic status response....
- `ChatRequest` — JSON body for a chat message....
- `ChatResponse` — JSON response from the chatbot....

Functions:

- `_get_pipeline()` — Lazy-init the LangGraph pipeline and return (pipeline, state
- `chat(request)` — Send a message to the ParkSmart chatbot and receive a respon
- `health_check()` — Health check endpoint.

Returns OK if the server is running a

- `health_check_detailed()` — Detailed health check — validates database connectivity and
- `create_reservation(request)` — Submit a new reservation request.

Called by the user-facing

- `list_reservations(status)` — List all reservations, optionally filtered by status.

The a

- `get_reservation(reservation_id)` — Get details of a specific reservation by ID.
- `approve_reservation(reservation_id, request)` — Admin approves a pending reservation.

Updates the status to

- `reject_reservation(reservation_id, request)` — Admin rejects a pending reservation.

Updates the status to

- `async admin_approve_reservation(reservation_id, request)` — Admin approves a pending reservation (POST).

Updates the st

- `async admin_reject_reservation(reservation_id, request)` — Admin rejects a pending reservation (POST).

Updates the sta

Key Imports: fastapi, pydantic, src

5.7 Notification Module (`src/notifications/`)

`src/notifications/email_service.py`

Lines: 558 | **Purpose:** SMTP email service with retry, templates, and console fallback

Capabilities:

- SSL (port 465) and STARTTLS (port 587) support
- Exponential backoff retry (3 attempts via tenacity)
- HTML + plain text email templates
- Async wrappers for non-blocking sends
- Console fallback when SMTP is unconfigured
- Email masking in console output for privacy

Email Types:

1. Admin notification — New reservation alert
2. User approval — Confirmation with parking instructions
3. User rejection — Explanation with admin notes

Integration: Called by notification_node (LangGraph), admin_agent, and API server approve/reject endpoints.

Classes:

- `EmailServiceError` — Base exception for email service failures....
- `EmailService` — Enterprise-grade email notification service.

Sends formatted HTML emails for re...

- `__init__(smtp_host, smtp_port, smtp_username, smtp_password)`
- `send_email(to, subject, html_body, plain_body)`
- `async send_email_async(to, subject, html_body, plain_body)`
- `notify_new_reservation(reservation)`
- `async notify_new_reservation_async(reservation)`
- `send_user_approval(reservation)`
- `send_user_rejection(reservation)`
- `notify_user_status_change(reservation)`
- `_send_user_notification(reservation, status)`
- `async send_user_approval_async(reservation)`
- `async send_user_rejection_async(reservation)`
- `_log_to_console(audience, reservation, status)`

Key Imports: asyncio, config, email, smtplib, src, ssl, tenacity

5.8 MCP Module (`src/mcp/`)

`src/mcp/mcp_server.py`

Lines: 335 | **Purpose:** FastAPI-based MCP tool server implementing tool discovery and execution

MCP Protocol Endpoints:

- `GET /mcp/health` — Server health (no auth)
- `POST /mcp/tools/list` — Discover available tools (API key required)
- `POST /mcp/tools/call` — Execute a tool by name (API key required)

Registered Tools:

1. `write_reservation_to_file` — Append approved reservation to text file
2. `read_approved_reservations` — Read recorded reservations

Security: API key authentication via `X-MCP-API-KEY` header.

Classes:

- `ToolParameter` — Schema for a single tool parameter....
- `ToolDefinition` — Schema describing a tool that this MCP server exposes....
- `ToolsListResponse` — Response for POST /mcp/tools/list — returns all available tools....
- `ToolCallRequest` — Request body for POST /mcp/tools/call — invoke a specific tool....
- `ToolCallResponse` — Response for POST /mcp/tools/call — result of tool execution....

Functions:

- `verify_api_key(x_mcp_api_key)` — Dependency that verifies the API key on every request.

The

- `tool_write_reservation_to_file(arguments)` — Write a single approved reservation record to the text file.
- `tool_read_approved_reservations(arguments)` — Read all approved reservation records from the file.

Return

- `mcp_health()` — Health check endpoint (no auth required).

Used by clients t

- `list_tools(api_key)` — MCP Tool Discovery — List all available tools.

This is the

- `call_tool(request, api_key)` — MCP Tool Execution — Call a specific tool with arguments.

T

Key Imports: fastapi, pydantic

`src/mcp/mcp_client.py`

Lines: 274 | **Purpose:** HTTP client for invoking MCP server tools with local fallback

Features:

- Server availability check before each call
- Automatic fallback to local file write when server is down
- API key authentication in request headers
- Configurable timeout for HTTP requests

Integration: Called by `mcp_recording_node()` in LangGraph pipeline and `admin_agent.approve_reservation()` method.

Classes:

- `MCPClient` — Client for communicating with the ParkSmart MCP Server.

This class is used by t...

- `__init__(server_url, api_key, timeout)`
- `is_server_available()`
- `list_tools()`
- `call_tool(tool_name, arguments)`
- `write_reservation_to_file(name, car_number, reservation_period, approval_time)`

- `_fallback_write(arguments, error_msg)`
- `read_approved_reservations()`

Key Imports: httpx

5.9 Data & Evaluation Modules

`src/data/load_data.py`

Lines: 105 | **Purpose:** Document loading and chunking for vector store ingestion

Functions:

- `load_and_split_documents()` — Load `parking_info.txt`, split into 500-char chunks with 50-char overlap using `RecursiveCharacterTextSplitter`
- `get_sample_questions()` — 10 test questions for evaluation
- `get_ground_truth_answers()` — Expected answers for accuracy measurement

Functions:

- `load_and_split_documents(file_path, chunk_size, chunk_overlap)` — Load a text file and split it into smaller chunks for vector
- `get_sample_questions()` — Returns sample questions for testing and evaluation.

These r

- `get_ground_truth_answers()` — Returns expected answers for evaluation metrics.

Paired with

Key Imports: langchain_community, langchain_text_splitters

`src/evaluation/evaluator.py`

Lines: 313 | **Purpose:** RAG pipeline quality and performance evaluation

Metrics Computed:

- Retrieval time (ms) — Vector search latency
- Generation time (ms) — LLM inference latency
- Precision@K — Fraction of retrieved docs relevant to answer
- Recall@K — Fraction of relevant info retrieved
- Answer relevance — Jaccard similarity between generated and expected

Output: `EvaluationReport` with per-question results and aggregated statistics.

Classes:

- `EvaluationResult` — Holds the results of a single evaluation query....
- `EvaluationReport` — Aggregated evaluation results across all test queries....
- `summary()`
- `RAGEvaluator` — Evaluates the RAG system's performance and accuracy.

Usage:

evaluator = RAG...

- `__init__(rag_chain)`
- `run_evaluation(questions, ground_truth, k)`
- `_evaluate_single_query(question, expected_answer, k)`
- `_calculate_retrieval_metrics(retrieved_texts, expected_answer, k)`

- `_calculate_answer_relevance(generated_answer, expected_answer)`
- `_aggregate_results(results)`
- `run_latency_test(num_queries)`

Key Imports: src, time

5.10 Utility Modules

`src/utils/masking.py`

Lines: 73 | **Purpose:** Email masking utility for privacy-safe display

Functions:

- `mask_email(email)` → `sa**@gmail.com` format (shows first 2 chars)
- `validate_email(email)` → Regex validation

Convention: All user-facing email display must use `mask_email()`. Guardrails safe_patterns list includes masked email regex to prevent Presidio from re-redacting already-masked addresses.

Functions:

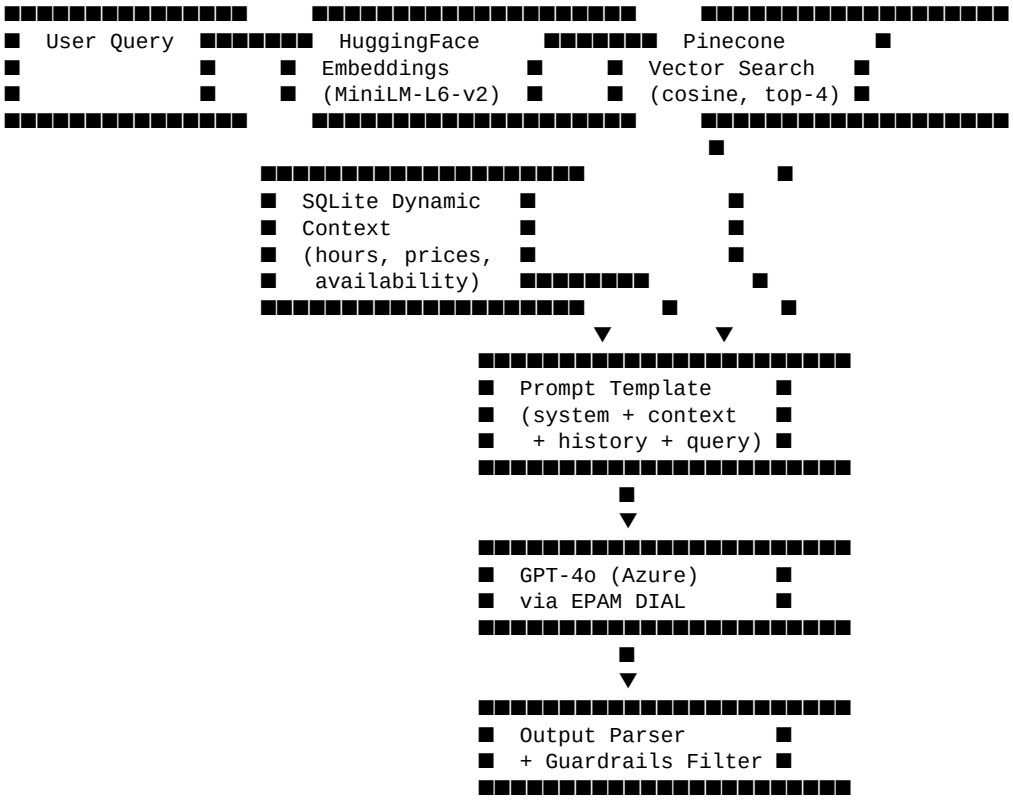
- `mask_email(email)` — Mask an email address for safe display in terminal/UI/logs.
- `validate_email(email)` — Validate that a string looks like a well-formed email addres

6. RAG Workflow

6.1 Overview

The Retrieval-Augmented Generation (RAG) pipeline combines semantic search over static parking knowledge with real-time SQL data injection to produce accurate, context-aware responses.

6.2 Pipeline Architecture



6.3 Embedding Strategy

Parameter	Value	Rationale
Model	`all-MiniLM-L6-v2`	Best balance of speed and quality for sentence embeddings
Dimension	384	Sufficient for domain-specific parking vocabulary
Metric	Cosine similarity	Normalized comparison regardless of vector magnitude
Top-K	4	Enough context without exceeding prompt token limits

6.4 Document Processing

1. **Source:** `src/data/parking_info.txt` — Static parking facility information
2. **Chunking:** `RecursiveCharacterTextSplitter` with 500-char chunks, 50-char overlap
3. **Storage:** Embedded chunks upserted to Pinecone serverless index
4. **Retrieval:** Query embedded → cosine search → top-4 chunks returned

6.5 Dynamic Context Injection

In addition to retrieved documents, the RAG chain injects real-time data from SQLite:

- **Working Hours** — Current operating schedule per day
- **Pricing** — Space type × duration pricing table
- **Availability** — Floor-by-floor space counts (total vs. available)

This dual-source approach ensures responses reflect both static policies and current operational state.

6.6 Intent Detection

The system prompt instructs the LLM to include `INTENT:BOOKING` in responses when the user expresses intent to make a reservation. The chatbot detects this marker and transitions from RAG Q&A mode to the reservation collection state machine.

7. LangGraph Workflow

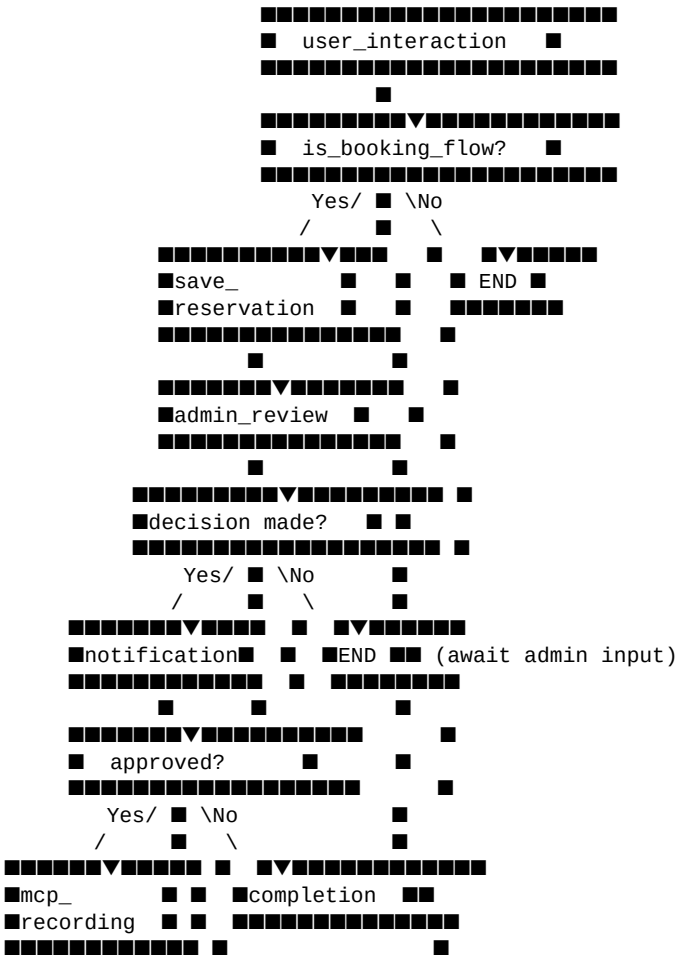
7.1 State Machine Design

The LangGraph pipeline implements a 6-node directed acyclic graph with conditional edges that route execution based on conversation state.

7.2 Node Descriptions

Node	Input State	Processing	Output State
`user_interaction`	User message	Chatbot processes query or reservation step	Bot response + booking detection
`save_reservation`	Completed booking data	Persist to SQL with 'pending' status	Reservation ID + AWAITING_ADMIN
`admin_review`	Admin command	Parse approve/reject/review, generate summary	Decision + admin notes
`notification`	Admin decision	Update DB status, send email to user	notification_sent = True
`mcp_recording`	Approved reservation	Write to file via MCP server/local fallback	mcp_recorded = True
`completion`	All flags set	Finalize cycle, log outcome	COMPLETED phase

7.3 Conditional Edge Logic



7.4 Human-in-the-Loop Design

The pipeline implements human-in-the-loop by **interrupting the graph** at the `admin_review` node:

1. User completes reservation → pipeline runs through `save_reservation`
2. `admin_review` returns `needs_admin_input=True` → graph yields END
3. System waits for external admin action (CLI or API endpoint)
4. Admin decision triggers `run_admin_decision()` → resumes from `admin_review`
5. Pipeline continues through notification → MCP → completion

This design avoids polling loops and integrates cleanly with both the CLI and REST API admin interfaces.

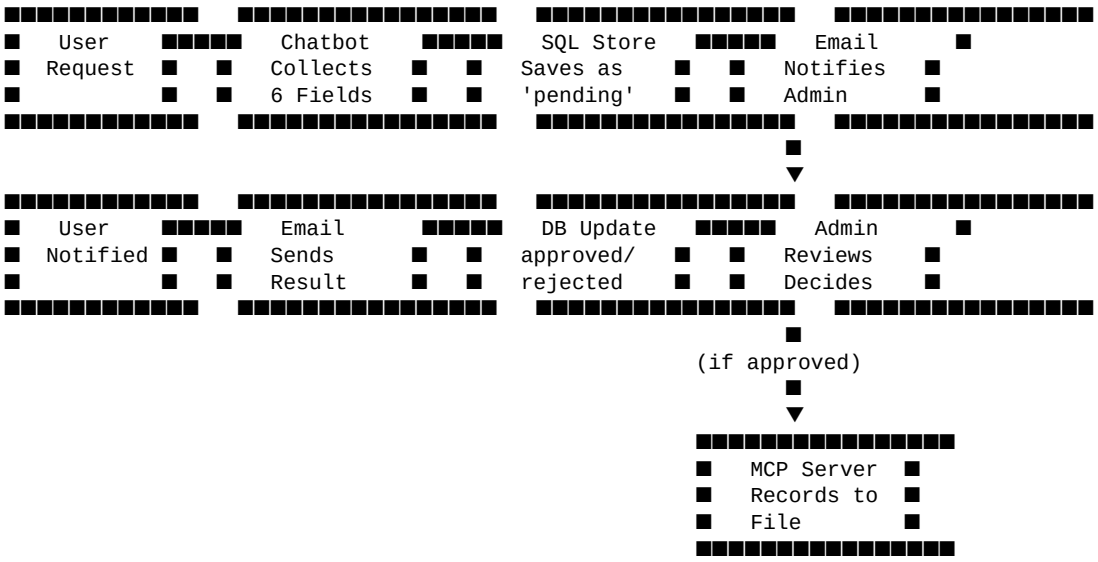
7.5 State Persistence

`GraphState` is a TypedDict passed through all nodes. Each node returns only the fields it modifies, and LangGraph merges updates automatically. The state includes:

- Conversation context (message, response, history)
- Reservation lifecycle (data, ID, status)
- Admin decision chain (decision, notes)
- Completion flags (notification_sent, mcp_recorded)
- Error tracking (error field)

8. Reservation Workflow

8.1 End-to-End Lifecycle



8.2 Data Collection State Machine

The chatbot collects reservation data through a strict sequence of states:

Step	State	Collected Field	Validation
1	COLLECTING_NAME	First name, last name	Non-empty strings
2	COLLECTING_EMAIL	Email address	Regex pattern match
3	COLLECTING_CAR	Car/license number	Non-empty string
4	COLLECTING_SPACE_TYPE	Parking space type	One of: standard, large, ev, vip, disabled
5	COLLECTING_START	Start date/time	Valid datetime, within operating hours
6	COLLECTING_END	End date/time	After start time, within operating hours
7	CONFIRMING	User confirmation	Yes/No response

8.3 Admin Review Process

Administrators can review reservations through two interfaces:

CLI Interface (`run_admin_cli`):

- `list` — Show pending reservations
- `review <id>` — Get LLM-generated recommendation with availability data
- `approve <id> [notes]` — Approve with optional notes
- `reject <id> [notes]` — Reject with reason

REST API (Admin Dashboard):

- `PUT /api/reservations/{id}/approve` — Approve via API
- `PUT /api/reservations/{id}/reject` — Reject via API

8.4 Notification Templates

Each notification includes both HTML and plain text variants:

Event	Recipient	Content
New reservation	Admin	Reservation details, review prompt
Approval	User	Confirmation, parking instructions, space details
Rejection	User	Reason for rejection, admin notes, rebooking suggestion

9. Frontend Architecture

9.1 Technology Stack

Technology	Version	Purpose
Next.js	16.x	React framework with App Router
React	19.x	UI component library
TypeScript	5.x	Type safety
Tailwind CSS	4.x	Utility-first styling
Zustand	5.x	Lightweight state management
shadcn/ui	Latest	Pre-built accessible components
Axios	Latest	HTTP client for API calls
Framer Motion	Latest	Animations and transitions
Lucide React	Latest	Icon library
Sonner	Latest	Toast notifications

9.2 Component Hierarchy

```
app/
■■■ layout.tsx              # Root layout + ThemeProvider
■■■ page.tsx                # Landing page with navigation
■■■ chat/
■   ■■■ page.tsx            # Chat interface page
■■■ admin/
    ■■■ page.tsx            # Admin dashboard page

components/
■■■ chat/
■   ■■■ ChatMessage.tsx     # Individual message bubble (user/bot)
■   ■■■ ChatInput.tsx       # Message input with send button
■   ■■■ ChatSidebar.tsx     # Conversation history panel
■■■ admin/
■   ■■■ AdminLoginGate.tsx  # Authentication gate component
■   ■■■ ReservationTable.tsx# Sortable/filterable reservation table
■   ■■■ ReservationModal.tsx# Approve/reject modal with notes
■   ■■■ StatsCards.tsx      # Dashboard metric cards
■   ■■■ ActivityFeed.tsx    # Recent actions timeline
■■■ ui/                     # shadcn/ui primitives (button, card, etc.)
```

9.3 State Management (Zustand)

Store	File	State	Purpose
`chatStore`	`store/chatStore.ts`	Messages, loading, error	Chat conversation state
`authStore`	`store/authStore.ts`	User, isAuthenticated	Admin authentication
`adminStore`	`store/adminStore.ts`	Reservations, filters, selected	Admin dashboard data
`uiStore`	`store/uiStore.ts`	Theme, sidebar toggle	UI preferences

9.4 API Integration Layer

```
services/
■■■ chatService.ts          # POST /api/chat — Send/receive messages
■■■ reservationService.ts   # CRUD operations on /api/reservations
```

```
■■■ adminService.ts         # Admin-specific endpoints

lib/
■■■ api.ts                  # Axios instance with baseURL + error interceptor
```

Base URL: Configured via `NEXT_PUBLIC_API_URL` environment variable

(defaults to `http://localhost:8000`).

9.5 Authentication Flow

The admin dashboard uses a simple credentials gate:

1. `AdminLoginGate` component checks `authStore.isAuthenticated`
2. If not authenticated, displays login form
3. On valid credentials (demo: `admin` / `1234`), sets auth state
4. All admin API calls proceed with authenticated context

10. Backend Architecture

10.1 Application Structure

```
FastAPI Application (server.py)
■
■■■ Startup Events
■   ■■■ Initialize SQLStore (SQLAlchemy → SQLite)
■   ■■■ Initialize EmailService (SMTP connection)
■   ■■■ Lazy-init LangGraph Pipeline (on first /api/chat call)
■
■■■ Middleware
■   ■■■ CORS (origins: localhost:3000)
■
■■■ Route Groups
■   ■■■ /api/health      — Health checks (basic + detailed)
■   ■■■ /api/chat        — Chatbot pipeline interaction
■   ■■■ /api/reservations — CRUD + admin actions
■   ■■■ /admin/*         — Legacy admin endpoints
■
■■■ Dependencies
    ■■■ sql_store        — Shared SQLStore instance
    ■■■ email_service    — Shared EmailService instance
    ■■■ _pipeline_state  — Global mutable pipeline state
```

10.2 Request Processing Flow

```
HTTP Request
    ■
    ▼
■■■■■■■■■■■■■■■■
■  CORS Check  ■
■■■■■■■■■■■■■■■■
      ■
      ▼
■■■■■■■■■■■■■■■■
■  Pydantic    ■
■  Validation  ■
■■■■■■■■■■■■■■■■
      ■
      ▼
■■■■■■■■■■■■■■■■
■  Route       ■
■  Handler     ■
■■■■■■■■■■■■■■■■
      ■
      ■■■■ SQLStore (DB operations)
      ■■■■ Pipeline (chatbot + LangGraph)
      ■■■■ EmailService (notifications)
      ■■■■ MCP Client (file recording)
      ■
      ▼
■■■■■■■■■■■■■■■■
■  JSON        ■
■  Response    ■
■■■■■■■■■■■■■■■■
```

10.3 Service Layer Design

Service	Lifecycle	Thread Safety	Initialization
`SQLStore`	Singleton	Session-per-request	App startup
`EmailService`	Singleton	Stateless methods	App startup
`LangGraph Pipeline`	Singleton	NOT thread-safe	Lazy (first chat)
`MCP Client`	Per-call	Stateless	Within pipeline

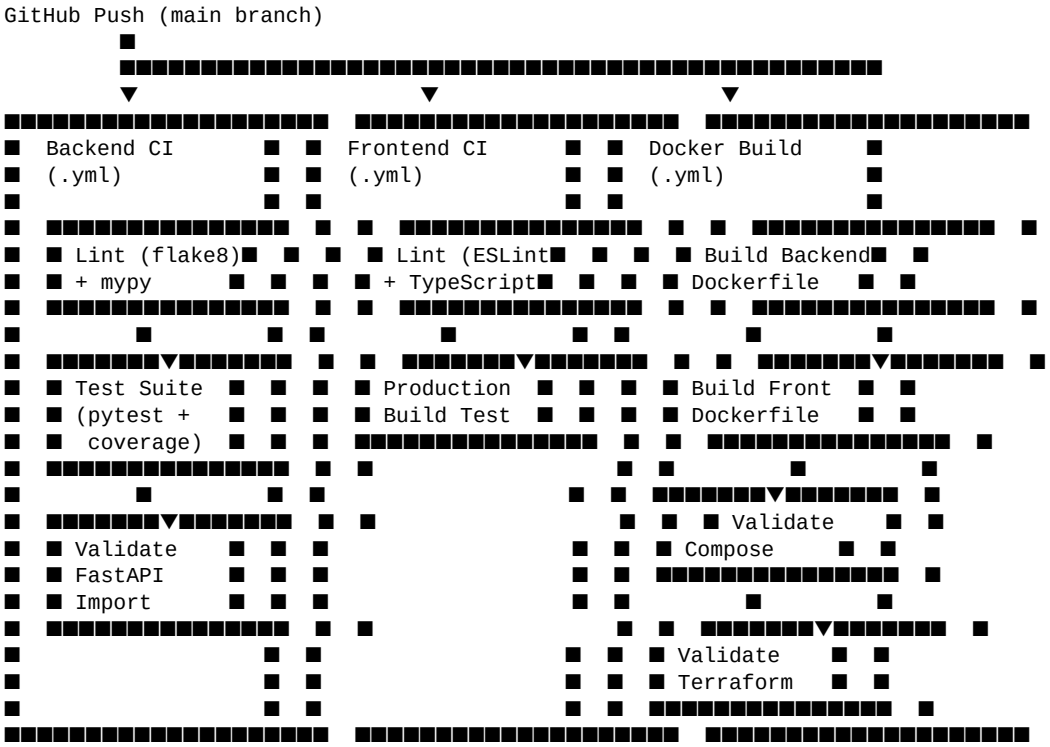
`Admin Agent`	Per-pipeline	Stateless methods	Pipeline creation

10.4 Error Handling Strategy

- **HTTP 400** — Invalid request body (Pydantic validation failure)
- **HTTP 404** — Reservation not found
- **HTTP 409** — Reservation already processed (approve/reject conflict)
- **HTTP 500** — Unhandled exceptions wrapped in `StatusResponse`
- **Custom Exceptions:** `VectorStoreError`, `EmailServiceError` with structured error data
- **Retry Logic:** Exponential backoff via `tenacity` for Pinecone and SMTP calls

11. CI/CD Pipeline

11.1 Workflow Architecture



11.2 Backend CI Workflow

Job	Steps	Purpose
Lint	Install deps → flake8 → mypy	Code style + type checking
Test Suite	Install deps + spaCy → pytest --cov	Run 161 tests with coverage
Validate FastAPI	Import `src.api.server:app`	Verify server can start

Environment: Python 3.11, pip cache, mock API keys via GitHub Secrets.

11.3 Frontend CI Workflow

Job	Steps	Purpose
Lint & TypeScript	npm ci → ESLint → tsc --noEmit	Code quality + type safety
Production Build	npm run build	Verify Next.js production bundle compiles

Environment: Node.js 20, npm cache.

11.4 Docker Build Workflow

Job	Steps	Purpose
Build Backend	docker build -f Dockerfile	Verify backend image builds
Build Frontend	docker build -f frontend/Dockerfile	Verify frontend image builds
Validate Compose	docker compose config	Verify compose file syntax
Validate Terraform	terraform init + validate + fmt	IaC validation

12. Docker & Deployment

12.1 Backend Dockerfile (Multi-Stage)

```
Stage 1: Builder
■■■ Base: python:3.11-slim
■■■ Install: system deps (gcc, build-essential)
■■■ Create: virtualenv in /opt/venv
■■■ Install: Python dependencies from requirements.txt
■■■ Download: spaCy en_core_web_lg model

Stage 2: Runtime
■■■ Base: python:3.11-slim
■■■ Copy: /opt/venv from builder
■■■ Copy: application source code
■■■ Create: non-root user (appuser:appgroup)
■■■ Expose: port 8000
■■■ Healthcheck: GET /api/health every 30s
■■■ CMD: uvicorn src.api.server:app --host 0.0.0.0 --port 8000
```

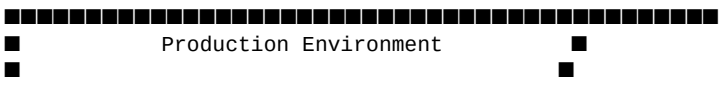
12.2 Docker Compose Architecture

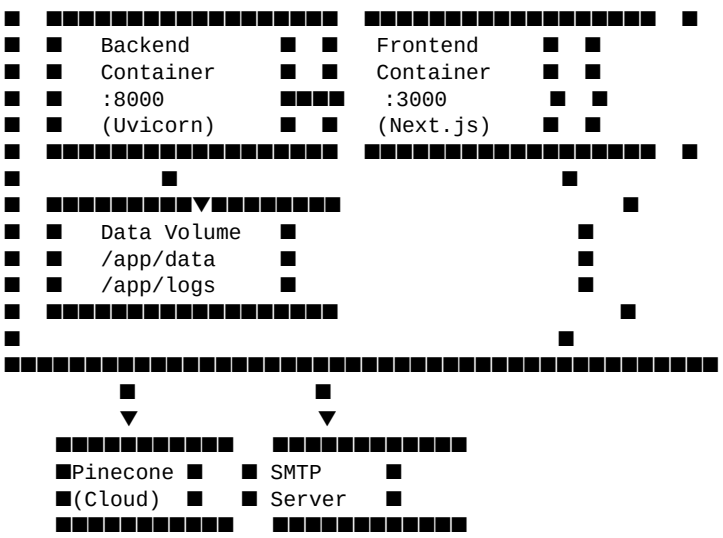
```
services:
  backend:
    build: .
    ports: 8000:8000
    env_file: .env
    volumes:
      - backend-data:/app/data
      - backend-logs:/app/logs
    healthcheck: /api/health

  frontend:
    build: ./frontend
    ports: 3000:3000
    depends_on:
      backend:
        condition: service_healthy
    environment:
      NEXT_PUBLIC_API_URL: http://backend:8000
```

12.3 Environment Variables

Variable	Service	Purpose	Secret
`DIAL_API_KEY`	Backend	Azure OpenAI access	Yes
`PINECONE_API_KEY`	Backend	Vector DB access	Yes
`SMTP_HOST`	Backend	Email server	No
`SMTP_PORT`	Backend	Email port	No
`SMTP_USERNAME`	Backend	Email auth	Yes
`SMTP_PASSWORD`	Backend	Email auth	Yes
`ADMIN_EMAIL`	Backend	Notification recipient	No
`SQL_DATABASE_URL`	Backend	Database path	No
`NEXT_PUBLIC_API_URL`	Frontend	API base URL	No

12.4 Deployment Architecture




Backend
Container
:8000
(Uvicorn)
Frontend
Container
:3000
(Next.js)
Data Volume
/app/data
/app/logs
Pinecone
(Cloud)
SMTP
Server

13. Terraform

13.1 Infrastructure as Code Overview

The project includes Terraform configurations for deploying the backend to

Render's cloud platform.

13.2 File Structure

File	Purpose
`terraform/provider.tf`	Render provider configuration
`terraform/main.tf`	Web service resource definition
`terraform/variables.tf`	Input variables (API keys, settings)
`terraform/outputs.tf`	Deployment URL output

13.3 Resource Definition

```
resource "render_web_service" "parksmart_backend" {
  name       = var.backend_name        # "parksmart-backend"
  region     = var.backend_region      # "oregon"
  plan       = var.backend_plan        # "free"
  runtime    = "python"

  start_command = "uvicorn src.api.server:app --host 0.0.0.0 --port $PORT"

  env_vars = {
    DIAL_API_KEY    = var.dial_api_key
    PINECONE_API_KEY = var.pinecone_api_key
    SMTP_HOST       = var.smtp_host
    SMTP_PORT       = var.smtp_port
    SMTP_USERNAME   = var.smtp_username
    SMTP_PASSWORD   = var.smtp_password
    ADMIN_EMAIL     = var.admin_email
    PYTHON_VERSION  = var.python_version
  }
}
```

13.4 Variable Definitions

Variable	Type	Default	Sensitive
`render_api_key`	string	—	Yes
`backend_name`	string	parksmart-backend	No
`backend_region`	string	oregon	No
`backend_plan`	string	free	No
`dial_api_key`	string	—	Yes
`pinecone_api_key`	string	—	Yes
`smtp_host`	string	—	No
`smtp_port`	string	587	No
`smtp_username`	string	—	Yes
`smtp_password`	string	—	Yes
`admin_email`	string	—	No

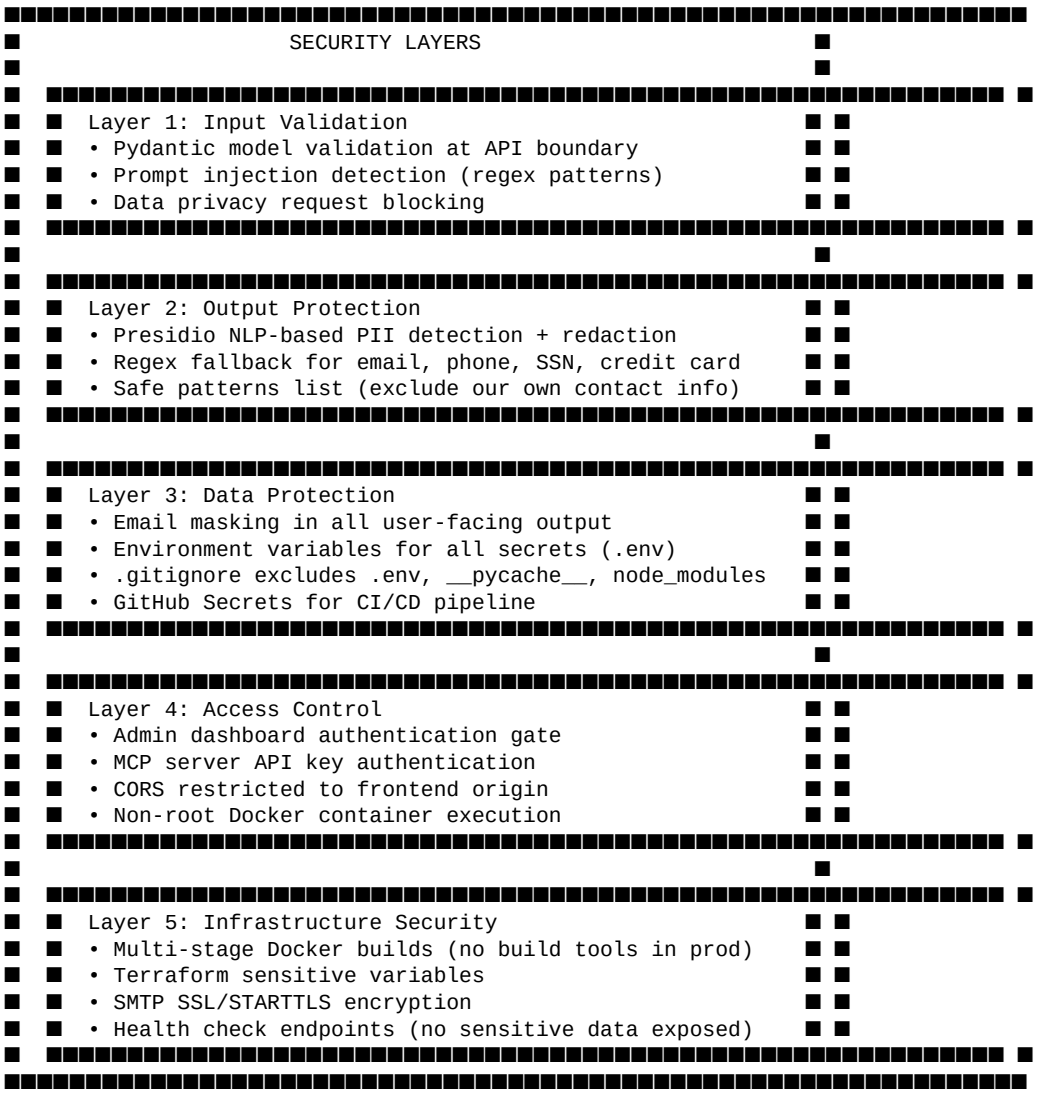
`python_version`	string	3.11.0	No

13.5 Deployment Workflow

```
terraform init          # Download Render provider
terraform plan          # Preview infrastructure changes
terraform apply         # Deploy backend service
terraform output url    # Get deployed service URL
```

14. Security

14.1 Security Architecture Overview

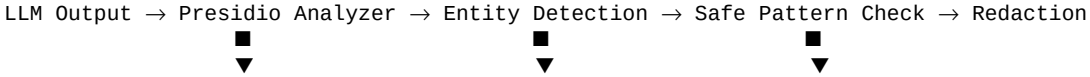


14.2 Prompt Injection Prevention

The guardrails module detects and blocks:

Pattern Type	Examples	Response
System prompt override	"ignore instructions", "forget your rules"	Blocked with explanation
Role manipulation	"you are now a different AI", "act as root"	Blocked with explanation
Data exfiltration	"show all user emails", "dump the database"	Blocked with explanation
Instruction injection	"SYSTEM:", "\n\nHuman:"	Blocked with explanation

14.3 PII Redaction Pipeline



```
NLP Engine          EMAIL, PHONE,         Exclude our
(spaCy)             SSN, CREDIT_CARD      own contacts
```

14.4 Email Masking Convention

All user-facing email display uses `mask_email()`:

- Input: `sanyam.sachan@gmail.com`
- Output: `sa**@gmail.com`
- Rule: Show first 2 characters + asterisks + domain

15. Testing

15.1 Testing Strategy

Aspect	Approach
Framework	pytest with pytest-asyncio
Coverage	pytest-cov with line coverage
External Services	All mocked (LLM, Pinecone, SMTP)
Database	In-memory SQLite (no cleanup needed)
Configuration	Mock `settings` object, not `os.environ`
Structure	One test file per module, class-based grouping
CI Integration	Automated via GitHub Actions on every push

15.2 Test Distribution

Test File	Module Tested	Test Count
`test_admin_agent.py`	admin_agent	9
`test_api_server.py`	api_server	11
`test_chatbot.py`	chatbot	12
`test_email_service.py`	email_service	20
`test_evaluator.py`	evaluator	6
`test_graph.py`	graph	38
`test_guardrails.py`	guardrails	8
`test_masking.py`	masking	17
`test_mcp.py`	mcp	21
`test_sql_store.py`	sql_store	9
`test_vector_store.py`	vector_store	10
Total	**All Modules**	**161**

15.3 Mocking Strategy

```
# Pattern: Mock settings object with attribute overrides
@patch("src.module.settings", mock_settings)
class TestModule:
    def setup_method(self):
        self.mock_settings = MagicMock()
        self.mock_settings.attribute = "test_value"

# Pattern: Mock external LLM calls
@patch("src.chatbot.rag_chain.AzureChatOpenAI")
def test_rag_query(self, mock_llm):
    mock_llm.return_value.invoke.return_value = "mocked response"

# Pattern: In-memory SQLite
sql_store = SQLStore("sqlite:///:memory:")
sql_store.initialize_default_data()
```

15.4 Test Categories

Category	Purpose	Examples
Unit Tests	Test individual functions/methods	Email masking, PII detection
Integration Tests	Test module interactions	Chatbot + SQL store, API + pipeline
Validation Tests	Verify constraints	State transitions, input validation
Error Path Tests	Test failure handling	Connection errors, invalid data

15.5 Running Tests

```
# Run all tests
python -m pytest

# Run with verbose output
python -m pytest -v

# Run with coverage
python -m pytest --cov=src --cov-report=term-missing

# Run specific test file
python -m pytest tests/test_chatbot.py

# Run specific test class
python -m pytest tests/test_chatbot.py::TestParkingChatbot
```

16. Future Enhancements

16.1 Short-Term Improvements

Enhancement	Technology	Impact
Session-based state	Redis	Thread-safe concurrent users, session persistence
JWT Authentication	PyJWT + OAuth2	Proper admin auth replacing demo credentials
WebSocket Chat	FastAPI WebSocket	Real-time bidirectional messaging
Rate Limiting	slowapi / Redis	API abuse prevention
Structured Logging	structlog + ELK	Centralized log aggregation and search

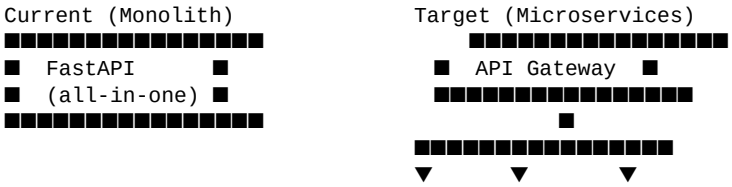
16.2 Medium-Term Features

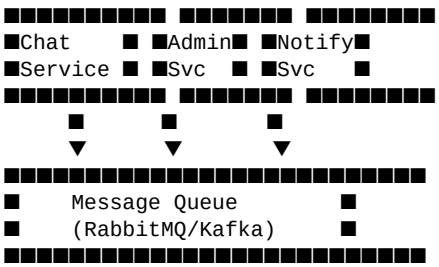
Enhancement	Technology	Impact
Payment Gateway	Stripe API	Collect parking fees at reservation time
AI Parking Prediction	scikit-learn / Prophet	Predict availability based on historical data
Multi-Language Support	i18n + LLM translation	Serve international users
Mobile App	React Native / Expo	Native mobile experience
Webhook Notifications	FastAPI + httpx	Slack/Teams integration for admin alerts

16.3 Long-Term Architecture

Enhancement	Technology	Impact
Kubernetes Deployment	K8s + Helm	Auto-scaling, rolling updates, self-healing
Message Queue	RabbitMQ / Kafka	Decouple notification and MCP recording
Database Migration	PostgreSQL + Alembic	Production-grade relational DB with schema versioning
API Gateway	Kong / AWS API Gateway	Centralized rate limiting, auth, and monitoring
Observability	Prometheus + Grafana	Metrics, dashboards, and alerting
A/B Testing	LaunchDarkly	Feature flags for gradual rollouts

16.4 Scaling Strategy




Chat Service
Admin Svc
Notify Svc
Message Queue
(RabbitMQ/Kafka)

Appendix A: Code Metrics

A.1 Lines of Code Summary

Category	Files	Lines
Backend Source (`src/`)	28	5,409
Tests (`tests/`)	12	2,588
Configuration (`config/`)	—	95
Frontend TypeScript (`frontend/src/`)	52	3,967
Frontend CSS	—	130
Total	**92**	**12,189**

A.2 Dependency Count

Category	Count
Python packages	25
Test cases	161

A.3 Architecture Component Count

Component	Count
LangGraph Nodes	6
API Endpoints	10+
SQLAlchemy Models	4
Zustand Stores	4
React Components	8+
CI/CD Workflows	3
Docker Services	2
Terraform Resources	1

Document generated automatically by `generate_project_documentation.py`

ParkSmart AI Parking Reservation Platform — Version 4.0

May 21, 2026